

PRÁCTICA 2 – RMI

Ibtissam El Amraoui El Houssaini y Candela Libertymeí López Grenville

1. DISEÑO DE LA INTERFAZ REMOTA

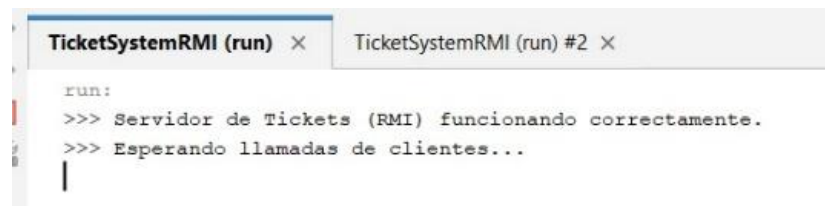
```
1 package com.tickets.common;
2
3 import java.rmi.Remote;
4 import java.rmi.RemoteException;
5 import java.util.List;
6 import java.util.Map;
7
8 /**
9  * Interfaz que define las operaciones del sistema de tickets.
10  * Sustituye al protocolo de texto (CREATE, GET, LIST...) de la P1.
11  */
12 public interface ITicketService extends Remote {
13
14     // CREATE -> crearTicket
15     String crearTicket(String user, String title, String desc, String priority) throws RemoteException;
16
17     // GET -> consultarTicket
18     Map<String, Object> consultarTicket(int id) throws RemoteException;
19
20     // LIST -> listarTickets
21     List<Integer> listarTickets(String estado, String asignado) throws RemoteException;
22
23     // STATE -> cambiarEstado
24     boolean cambiarEstado(int id, String nuevoEstado) throws RemoteException;
25
26     // ASSIGN/REASSIGN -> asignarTicket
27     boolean asignarTicket(int id, String responsable) throws RemoteException;
28
29     // COMMENT -> añadirComentario
30     void añadirComentario(int id, String user, String texto) throws RemoteException;
31 }
```

La interfaz ITicketService define las operaciones remotas del sistema de tickets. Esta interfaz funciona como contrato entre cliente y servidor: el cliente conoce qué métodos puede invocar, pero no necesita saber cómo están implementados internamente.

Los mensajes del protocolo anterior se han transformado en métodos RMI. Por ejemplo, CREATE pasa a ser crearTicket(), GET pasa a ser consultarTicket(), LIST pasa a ser listarTickets() y así con el resto de operaciones.

Además, la interfaz extiende Remote y todos los métodos lanzan RemoteException, ya que las llamadas se realizan a través de la red y pueden producirse fallos de comunicación. La implementación real de estas operaciones se encuentra en TicketServiceImpl, que es la clase ejecutada en el servidor.

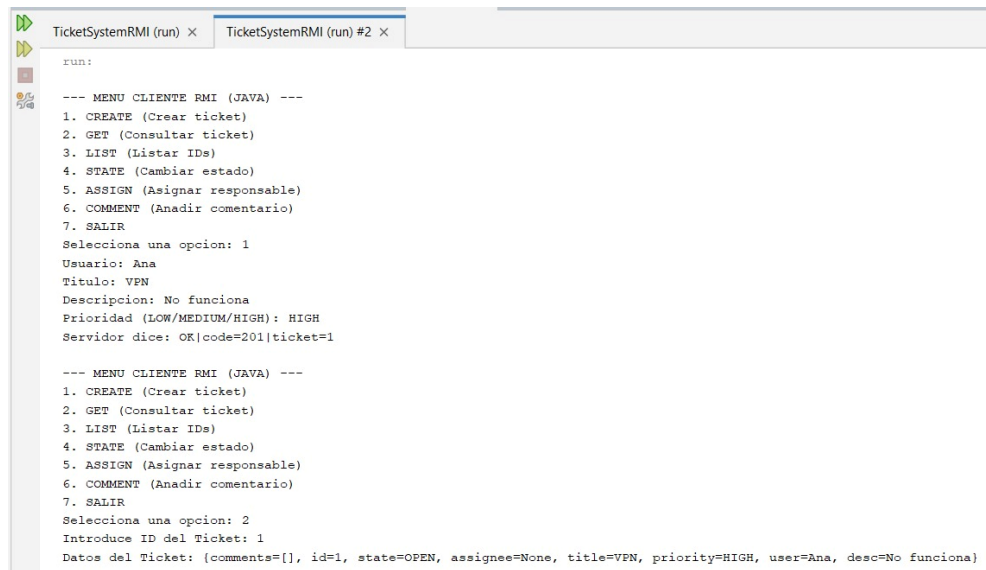
2. EJECUCIÓN DEL SERVIDOR



```
run:
>>> Servidor de Tickets (RMI) funcionando correctamente.
>>> Esperando llamadas de clientes...
|
```

En esta captura se muestra la ejecución del servidor RMI. El servidor queda a la espera de llamadas de los clientes para gestionar las operaciones sobre los tickets.

3. CREACIÓN Y CONSULTA DE TICKETS



```
run:
--- MENU CLIENTE RMI (JAVA) ---
1. CREATE (Crear ticket)
2. GET (Consultar ticket)
3. LIST (Listar IDs)
4. STATE (Cambiar estado)
5. ASSIGN (Asignar responsable)
6. COMMENT (Anadir comentario)
7. SALIR
Selecciona una opcion: 1
Usuario: Ana
Titulo: VPN
Descripcion: No funciona
Prioridad (LOW/MEDIUM/HIGH): HIGH
Servidor dice: OK|code=201|ticket=1

--- MENU CLIENTE RMI (JAVA) ---
1. CREATE (Crear ticket)
2. GET (Consultar ticket)
3. LIST (Listar IDs)
4. STATE (Cambiar estado)
5. ASSIGN (Asignar responsable)
6. COMMENT (Anadir comentario)
7. SALIR
Selecciona una opcion: 2
Introduce ID del Ticket: 1
Datos del Ticket: {comments=[], id=1, state=OPEN, assignee=None, title=VPN, priority=HIGH, user=Ana, desc=No funciona}
```

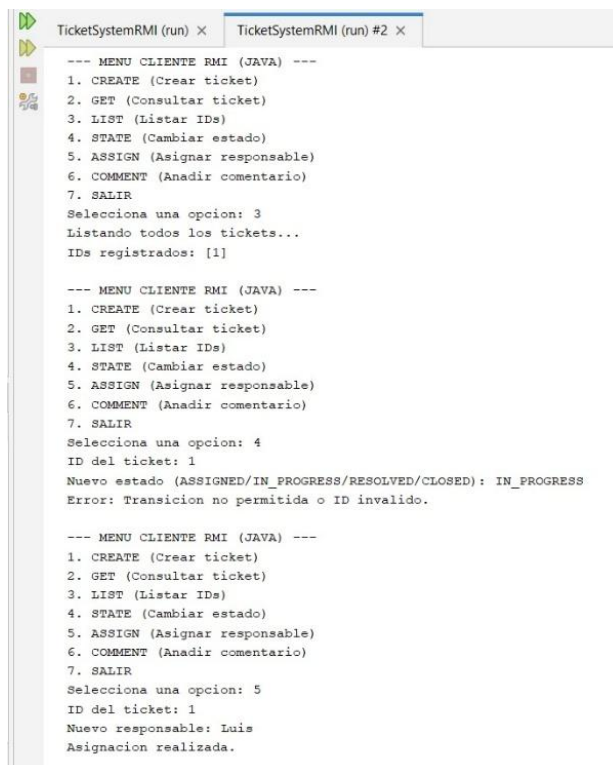
En esta ejecución el cliente crea un nuevo ticket indicando usuario, título, descripción y prioridad. Posteriormente se consulta el ticket creado, mostrando su información completa (estado, usuario, prioridad, etc.). Estas operaciones se realizan mediante invocaciones remotas al servidor.

4. LISTADO, ASIGNACIÓN Y CONTROL DE ERRORES

En la siguiente captura se muestran varias operaciones del sistema:

- Listado de tickets registrados
- Intento de cambio de estado con error (validación del servidor)
- Asignación de responsable a un ticket

Esto demuestra la gestión de lógica y validación en el servidor remoto.



```
TicketSystemRMI (run) x TicketSystemRMI (run) #2 x

--- MENU CLIENTE RMI (JAVA) ---
1. CREATE (Crear ticket)
2. GET (Consultar ticket)
3. LIST (Listar IDs)
4. STATE (Cambiar estado)
5. ASSIGN (Asignar responsable)
6. COMMENT (Anadir comentario)
7. SALIR
Selecciona una opcion: 3
Listando todos los tickets...
IDs registrados: [1]

--- MENU CLIENTE RMI (JAVA) ---
1. CREATE (Crear ticket)
2. GET (Consultar ticket)
3. LIST (Listar IDs)
4. STATE (Cambiar estado)
5. ASSIGN (Asignar responsable)
6. COMMENT (Anadir comentario)
7. SALIR
Selecciona una opcion: 4
ID del ticket: 1
Nuevo estado (ASSIGNED/IN_PROGRESS/RESOLVED/CLOSED): IN_PROGRESS
Error: Transicion no permitida o ID invalido.

--- MENU CLIENTE RMI (JAVA) ---
1. CREATE (Crear ticket)
2. GET (Consultar ticket)
3. LIST (Listar IDs)
4. STATE (Cambiar estado)
5. ASSIGN (Asignar responsable)
6. COMMENT (Anadir comentario)
7. SALIR
Selecciona una opcion: 5
ID del ticket: 1
Nuevo responsable: Luis
Asignacion realizada.
```

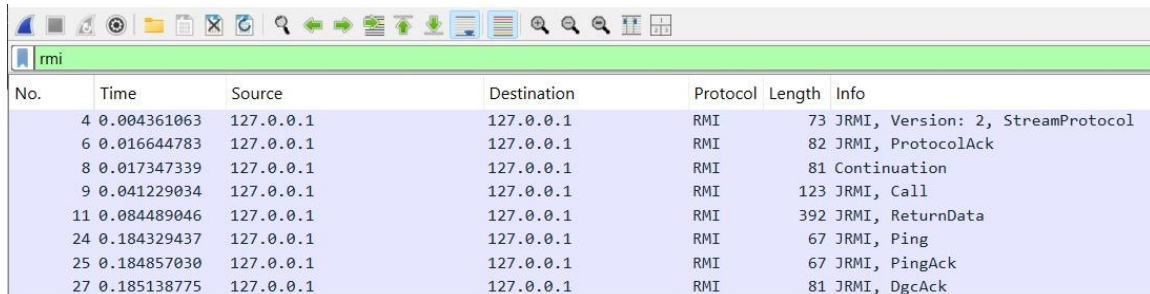
5. AÑADIR COMENTARIOS

En esta ejecución se añade un comentario a un ticket existente. El cliente envía la petición y el servidor la procesa correctamente, confirmando la operación

```
--- MENU CLIENTE RMI (JAVA) ---
1. CREATE (Crear ticket)
2. GET (Consultar ticket)
3. LIST (Listar IDs)
4. STATE (Cambiar estado)
5. ASSIGN (Asignar responsable)
6. COMMENT (Añadir comentario)
7. SALIR
Selecciona una opcion: 6
ID del ticket: 1
Tu nombre/usuario: Luis
Mensaje del comentario: En proceso
Comentario añadido correctamente.

--- MENU CLIENTE RMI (JAVA) ---
1. CREATE (Crear ticket)
2. GET (Consultar ticket)
3. LIST (Listar IDs)
4. STATE (Cambiar estado)
5. ASSIGN (Asignar responsable)
6. COMMENT (Añadir comentario)
7. SALIR
Selecciona una opcion: 7
Saliendo del sistema...
BUILD SUCCESSFUL (total time: 1 minute 5 seconds)
```

6. CAPTURA WIRESHARK (RMI)

A screenshot of the Wireshark network protocol analyzer interface. The top toolbar shows various icons for file operations, editing, and analysis. Below the toolbar, the 'rmi' filter is applied to the packet list. The packet list table shows 10 packets, all of which are RMI protocol messages between 127.0.0.1 and 127.0.0.1. The 'Info' column provides details for each packet, such as 'JRMII, Version: 2, StreamProtocol', 'JRMII, ProtocolAck', 'Continuation', 'JRMII, Call', 'JRMII, ReturnData', 'JRMII, Ping', 'JRMII, PingAck', and 'JRMII, DgcAck'.

No.	Time	Source	Destination	Protocol	Length	Info
4	0.004361063	127.0.0.1	127.0.0.1	RMI	73	JRMII, Version: 2, StreamProtocol
6	0.016644783	127.0.0.1	127.0.0.1	RMI	82	JRMII, ProtocolAck
8	0.017347339	127.0.0.1	127.0.0.1	RMI	81	Continuation
9	0.041229034	127.0.0.1	127.0.0.1	RMI	123	JRMII, Call
11	0.084489046	127.0.0.1	127.0.0.1	RMI	392	JRMII, ReturnData
24	0.184329437	127.0.0.1	127.0.0.1	RMI	67	JRMII, Ping
25	0.184857030	127.0.0.1	127.0.0.1	RMI	67	JRMII, PingAck
27	0.185138775	127.0.0.1	127.0.0.1	RMI	81	JRMII, DgcAck

En esta captura se muestra el tráfico filtrado por el protocolo RMI. Se observan paquetes como:

- **Call** (innovación de método remoto)
- **ReturnData** (respuesta del servidor)
- **ProtocoloAck, Ping, etc.**

Esto refleja la comunicación entre el cliente y servidor mediante invocaciones remotas.

Aun así, aparecen nuevos problemas propios de la invocación remota. El cliente y el servidor deben compartir correctamente la interfaz remota, y todos los métodos deben controlar posibles `RemoteException`, ya que pueden producirse errores de red, fallos del servidor o problemas al localizar el servicio. Además, aunque las llamadas parezcan locales, realmente se ejecutan a través de la red, por lo que puede existir latencia. También es necesario arrancar primero el servidor y registrar correctamente el servicio RMI antes de ejecutar el cliente.